

Submission Worksheet

CLICK TO GRADE

<https://learn.ethereallab.app/assignment/IT202-007-F2024/it202-module-6-milestone-1-2024/grade/par36>

Course: IT202-007-F2024

Assignment: [IT202] Module 6 Milestone 1 2024

Student: Peter R. (par36)

Submissions:

Submission Selection

1 Submission [submitted] 11/9/2024 5:20:34 PM

Instructions

^ COLLAPSE ^

Overview Video: <https://youtu.be/V7oHa8KKtss>

Prereqs:

- Go through each lesson from "Project Setup and SQL" to "User Login Enhancement" and follow the branching names while gathering the code
- Merge each into Milestone1 branch
- Create a Project board on GitHub (if you haven't yet)
 - Add each major item from the proposal doc as an Issue item
 - Invite the grader(s) and myself as collaborators on the board (they're separate from your repository)
 - ☐ See Canvas announcements for the Usernames or check your collab list on your repo
- Mark the related GitHub Issues items as "done"
- Implement your own custom CSS (something much different than the default "ugly" CSS given as an example)
- Consider styling all forms/inputs, data output, navigation, etc
- Implement JavaScript validation on Register, Login, and Profile (include "[Client]" in the output messages to differentiate between server-side validations)

Instructions:

1. Make sure you're in Milestone1 with the latest changes pulled
2. Ensure Milestone1 has been deployed to heroku dev
3. Gather the requested evidence and fill in the explanations per each prompt
4. Save the submission and generate the output PDF

5. Put the output PDF into your local repository folder
6. add/commit/push it to GitHub
7. Merge Milestone1 into dev
8. Locally checkout dev and pull the changes
9. Create and merge a pull request from dev to prod to deploy Milestone1 to prod
10. Upload this output PDF to Canvas

Branch name: Milestone1

Group

100%

Group: User Registration

Tasks: 6

Points: 2

^ COLLAPSE ^

Task

100%

Group: User Registration

Task #1: Screenshot of form on website page

Weight: ~17%

Points: ~0.33

^ COLLAPSE ^

Details:

Thoughtful CSS should be applied to all parts (must differ from the "ugly" CSS given via the lessons). The Heroku dev URL must be present in all screenshots of the site.



Columns: 4

Sub-Task

100%

Group:
User
Registration
Task #1:
Screenshot
of form on
website

Sub-Task

100%

Group:
User
Registration
Task #1:
Screenshot
of form on
website

Sub-Task

100%

Group:
User
Registration
Task #1:
Screenshot
of form on
website

Sub-Task

100%

Group:
User
Registration
Task #1:
Screenshot
of form on
website



Task
Screenshots

Gallery Style: 2 Columns

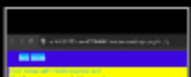
4 2 1



Task
Screenshots

Gallery Style: 2 Columns

4 2 1



Task
Screenshots

Gallery Style: 2 Columns

4 2 1



Task
Screenshots

Gallery Style: 2 Columns

4 2 1





Screenshot of the Register form on Heroku dev with the correct information filled in and working.

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown



Shows validations for each field

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown



shows a message when the email is already taken

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown



Shows a message when a username is already taken

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

Task URLs

URL #1

<https://par36-it202-007-prod-d727c5625e8e.herokuapp.com/project/register.php>

URL

[https://par36-it202-](https://par36-it202-007-prod-d727c5625e8e.herokuapp.com/project/register.php)

Sub-Task

100%

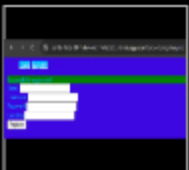
Group:
User
Registration
Task #1:
Screenshot
of form on
website



Task Screenshots

Gallery Style: 2 Columns

4 2 1



Shows the user they have successfully registered

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

Task

100%

Group: User Registration
Task #2: Screenshot of the form code
Weight: ~17%
Points: ~0.33

^ COLLAPSE ^

Details:

Should have the appropriate type attributes for the fields.
Include uid/date code comments in all screenshots (one per screenshot is sufficient)



Sub-Task

100%

Group: User Registration
Task #2: Screenshot of the form code
Sub Task #1: Show the html of the form

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
<form onSubmit="return validate(this)" method="POST" id="page-1-submit-form" data-bbox="70 494 523 653">
  <div>
    <label for="email">Email</label>
    <input type="email" name="email" required />
  </div>
  <div>
    <label for="username">Username</label>
    <input type="text" name="username" required minlength="3" />
  </div>
  <div>
    <label for="password">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <div>
    <label for="confirm">Confirm</label>
    <input type="password" name="confirm" required minlength="8" />
  </div>
  <input type="submit" value="Register" />
</form>
```

The HTML code takes in user data

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Briefly explain the html for each field including the chosen attributes

Response:

The first line takes in the information (if it doesn't have any errors) and send it to the Users database where the information is stored. The next many lines contain the parameters for the input data and defines them accordingly. It also sets the minimum and maximum length for certain types of inputs such as the password input and the username input (respectively).

End of Task 2

Task

100%

Group: User Registration

Task #3: Screenshot of the client-side and server-side validation code

Weight: ~17%

Points: ~0.33

^ COLLAPSE ^

Details:

Include ucid/date code comments in all screenshots (one per screenshot is sufficient)



Columns: 2

Sub-Task

100%

Group: User Registration

Task #3: Screenshot of the client-side and server-side validation code

Sub Task #1: Show the JavaScript validations of the form (include any extra files related if you made separate files)

Sub-Task

100%

Group: User Registration

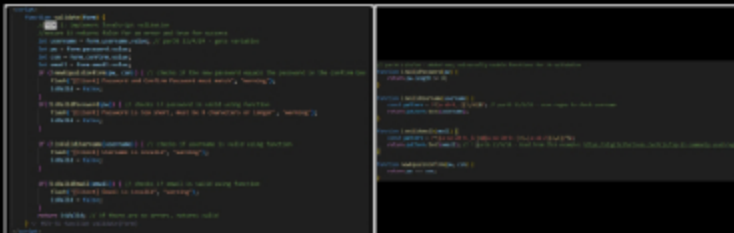
Task #3: Screenshot of the client-side and server-side validation code

Sub Task #2: Show the PHP validations (include any lib content)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Uses functions from helpers.js to do Client-Side validation

shows helpers.js functions

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Explain in concise steps how this logically works

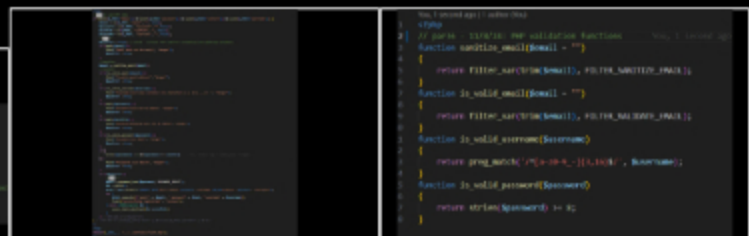
Response:

This defines variables for each type of relevant input in the form in the first few lines and then uses them in the functions which come from helpers.js. It checks against the conditions to see if they are not true and from there, if the conditions (as specified in helpers.js) are violated isValid is set to false. This prints out the JavaScript client side message.

Task Screenshots

Gallery Style: 2 Columns

4 2 1



PHP code for Server-Side validation

This code in sanitizers.php validates the code

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Explain in concise steps how this logically works

Response:

The PHP code also defines/clears out variables in its first few lines and also gets functions to determine the course of action from a user's input. The main difference besides it being in PHP/server side is that it gets its functions from a different file (sanitizers.php) and from there it checks against the conditions and if they aren't met it prints out a PHP flash message and make the user enter the info the right way, just like with JavaScript validation.

End of Task 3

Task



Group: User Registration

Task #4: Screenshot of the Users table with a valid user entry

Weight: ~17%

Points: ~0.33

^ COLLAPSE ^

Checklist

*The checkboxes are for your own tracking

#	Details
☐ #1	Password should be hashed
☐ #2	Should have email, password, username (unique), created, modified, and id fields
☐ #3	Ensure left panel or database name is present (should contain your ucid)

Sub-Task



Group: User Registration

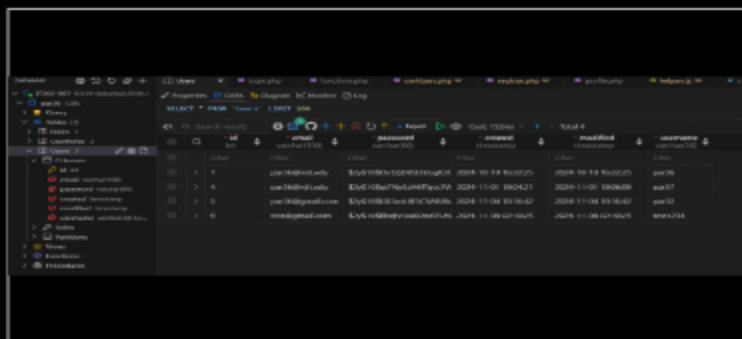
Task #4: Screenshot of the Users table with a valid user entry

Sub Task #1: Show valid data per the checklist

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Users table with many users, their usernames, IDs and hashed passwords (among other things).

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 4

Task



Group: User Registration

Task #5: Explain the registration logic in a step-by-step manner from when the page loads to when the data is saved to the DB

Weight: ~17%

Points: ~0.33

^ COLLAPSE ^

Details:

Don't just show code, translate things to plain English in concise steps.

May be worthwhile using a list.



Task Response Prompt

Response:

1. The form on the website has defined attributes in the code.
2. A user enters their inputs into those attributes.
3. Assuming the form is filled out correctly, the variables for those attributes are sent into the database through the end of the PHP statement. One notable difference is before the password is sent into the database, it is taken and hashed by the password_hash function for extra security.
4. Now with all of the data loaded into the database, a user can login and assuming they enter the correct credentials their account will work.

End of Task 5

Task



Group: User Registration

Task #6: Include pull request links related to this feature

Weight: ~17%

Points: ~0.33

 COLLAPSE 

Details:

Should end in /pull/#



Task URLs

URL #1

<https://github.com/PeterRedfern/par36-it202-007/pull/15>

URL

<https://github.com/PeterRedfern/par36-it202-007>

URL #2

<https://github.com/PeterRedfern/par36-it202-007/pull/34>

URL

<https://github.com/PeterRedfern/par36-it202-007>

End of Task 6

End of Group: User Registration

Task Status: 6/6

Group

Group: User Login

100%

Tasks: 4
Points: 2

^ COLLAPSE ^

Task

100%

Group: User Login
 Task #1: Screenshot of form on website page
 Weight: ~25%
 Points: ~0.50

^ COLLAPSE ^

Details:

Thoughtful CSS should be applied to all parts (must differ from the "ugly" CSS given via the lessons).
 The Heroku dev URL must be present in all screenshots of the site.



Columns: 3

Sub-Task

100%

Group: User Login
 Task #1:
 Screenshot of form
 on website page
 Sub Task #1: Two
 Screenshot of the
 form (one with

Sub-Task

100%

Group: User Login
 Task #1:
 Screenshot of form
 on website page
 Sub Task #2:
 Demonstrate
 JavaScript

Sub-Task

100%

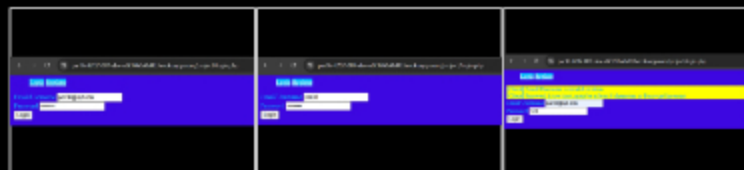
Group: User Login
 Task #1:
 Screenshot of form
 on website page
 Sub Task #3:
 Demonstrate user-
 friendly message



Task Screenshots

Gallery Style: 2 Columns

4 2 1



The login box with the email login being used/URL gone https://par36-it202-007-prod-

The login box with the username login being used

shows the client the email and password are invalid

d727c5625e8e.herokuapp.com/project/

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task URLs



Task Screenshots

Gallery Style: 2 Columns

4 2 1



Task Screenshots

Gallery Style: 2 Columns

4 2 1



Shows the user that the account isn't correct

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Sub-Task

Group: User Login
 Task #1:

Sub-Task

Group: User Login
 Task #1:

Sub-Task

Group: User Login
 Task #1:

100%

Task #1:
Screenshot of form
on website page
Sub Task #4:
Demonstrate user-
friendly message

100%

Task #1:
Screenshot of form
on website page
Sub Task #5:
Demonstrate
successful login

100%

Task #1:
Screenshot of form
on website page
Sub Task #6:
Demonstrate
session data being

Task Screenshots

Gallery Style: 2 Columns

4 2 1



shows the user that
the password is
incorrect or too
short

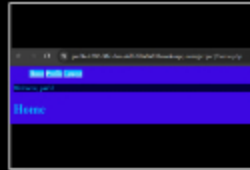
Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's
being shown*

Task Screenshots

Gallery Style: 2 Columns

4 2 1



the homepage with
the user's ID

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's
being shown*

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Shows the session
data being loaded
in via the logs

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's
being shown*

End of Task 1

Task



Group: User Login
Task #2: Screenshot of the form code
Weight: ~25%
Points: ~0.50

^ COLLAPSE ^

Details:

Should have the appropriate type attributes for the fields.
Include uid/date code comments in all screenshots (one per screenshot is sufficient)



Columns: 3

Sub-Task

100%

Group: User Login
Task #2: Screenshot
of the form code
Sub Task #1: Show
the html of the form

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Sub-Task

100%

Group: User Login
Task #2:
Screenshot of the
form code
Sub Task #2: Show
the JavaScript
validations of the

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Sub-Task

100%

Group: User Login
Task #2:
Screenshot of the
form code
Sub Task #3: Show
PHP validations
(include any lib

Task Screenshots

Gallery Style: 2 Columns

4 2 1



The HTML code from login.php

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response

Prompt

Briefly explain the html for each field including the chosen attributes

Response:

The first link checks the input when sent against what is in the database to see if it is a match. Below that, it has the layout for the Email/Username and Password boex as well as the input types and the submit button.



Direct JS code in login.php



The functions called in the first picture that are located in helpers.js

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response

Prompt

Explain in concise steps how this logically works

Response:

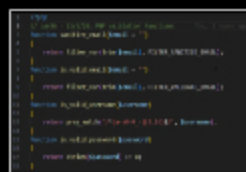
Login.php uses the same functions from helpers.js, just like register.php. However, this time it is changed as email and username are under a single field. While trying to code to differentiate them, the result didn't work out perfectly. Instead of the code picking up on the @ symbol and printing out "email invalid", it would always default to username (even though the code looks correct) so the all in one (Email/Username is incorrect) is a workaround to code that should work (in theory). The password part works as intended and detects if the password is too short or incorrect in one message.



PHP code in login.php (part 1)



PHP code in login.php (part 2)



sanitizers.php (from the lib folder).

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response

Prompt

Explain in concise steps how this logically works

Response:

In this example, the first few lines set up the inputs to be checked against the databse. From there, conditions are defines that would catch the input as invalid and make the user try to login again the correct way.

End of Task 2

Task



Group: User Login

Task #3: Explain the login logic step-by-step from when the page loads to when the data is fetched from the DB and stored in the session

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

Details:

Don't just show code, translate things to plain English in concise steps.
Explain how the session works and why/how it's used



May be worthwhile using a list.

Task Response Prompt

Response:

1. The login/password forms have defined attributes in the code.
2. A user enters the username/email and their password into those the form.
3. The request checks against the information in the database and a line of code at the end also pverifies the password with password_verify by checking the hash of the user input against the hash stored in the database.
4. Once the information is confirmed to be correct, the user will have access to their account on the website.

End of Task 3

Task



Group: User Login

Task #4: Include pull request links related to this feature

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

Details:

Should end in /pull/#



Task URLs

URL #1

<https://github.com/PeterRedfern/par36-it202-007/pull/29>

URL

<https://github.com/PeterRedfern/par36-it202-007>

URL #2

<https://github.com/PeterRedfern/par36-it202-007/pull/53>

URL

<https://github.com/PeterRedfern/par36-it202-007>

End of Task 4

End of Group: User Login

Task Status: 4/4

Group



Group: User Logout

Tasks: 2

Points: 1

^ COLLAPSE ^

Task



Group: User Logout
Task #1: Capture the following screenshots
Weight: ~50%
Points: ~0.50

^ COLLAPSE ^

Columns: 3

Sub-Task



Group: User Logout
Task #1: Capture the following screenshots
Sub Task #1:
Screenshot of the navigation when

Sub-Task



Group: User Logout
Task #1: Capture the following screenshots
Sub Task #2:
Screenshot of the redirect to login

Sub-Task



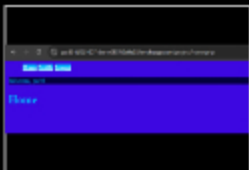
Group: User Logout
Task #1: Capture the following screenshots
Sub Task #3:
Screenshot of the logout-related code



Task Screenshots

Gallery Style: 2 Columns

4 2 1



logged in screen

Caption(s) (required) ✓

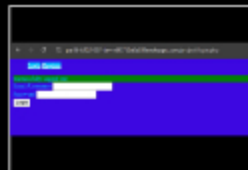
Caption Hint: *Describe/highlight what's being shown*



Task Screenshots

Gallery Style: 2 Columns

4 2 1



User gets logged out and redirected to login.php

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*



Task Screenshots

Gallery Style: 2 Columns

4 2 1



logout.php

reset session code

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain in concise steps how this logically works

Response:

When the user clicks on the logout button, logout.php calls in the function reset_session and reset_session unsets the session, destroys it and then starts a new one. From there, the user is redirected to login.php where the friendly logout message is displayed.

End of Task 1

Task



Group: User Logout

Task #2: Include pull request links related to this feature

Weight: ~50%

Points: ~0.50

^ COLLAPSE ^

Details:

Should end in /pull/#



Task URLs

URL #1

<https://github.com/PeterRedfern/par36-it202-007/pull/29>

URL

<https://github.com/PeterRedfern/par36-it202-007>

URL #2

<https://github.com/PeterRedfern/par36-it202-007/pull/31>

URL

<https://github.com/PeterRedfern/par36-it202-007>

End of Task 2

End of Group: User Logout

Task Status: 2/2

Group



Group: Basic Security Rules and Roles

Tasks: 5

Points: 2

^ COLLAPSE ^

Task



Group: Basic Security Rules and Roles

Task #1: Authentication Screenshots

Weight: ~20%

Points: ~0.40

^ COLLAPSE ^

Details:

Include ucid/date code comments in all screenshots (one per screenshot is sufficient)



Columns: 3

Sub-Task

100%

Group: Basic
Security Rules and
Roles
Task #1:
Authentication
Screenshots
Sub Task #1:

Sub-Task

100%

Group: Basic
Security Rules and
Roles
Task #1:
Authentication
Screenshots
Sub Task #2:

Sub-Task

100%

Group: Basic
Security Rules and
Roles
Task #1:
Authentication
Screenshots
Sub Task #3:



Task Screenshots

Gallery Style: 2 Columns

4 2 1



This function in
user_helpers.php
checks if a user is
logged in or not.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's
being shown*

Task Response Prompt

*Explain in concise steps how this
logically works*

Response:

This code checks if a session is set
and if a user is logged in. If a user
isn't logged in, it redirects them to
the login page and prints a message
telling them they must be logged in
to view whatever page they were
trying to access.



Task Screenshots

Gallery Style: 2 Columns

4 2 1



This is a
screenshot of
is_logged_in being
used in profile.php
(profile page), it is
also used in the
home page
(home.php).

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's
being shown*

Task Response Prompt

*Explain in concise steps how this
logically works*

Response:

The code fetches is_logged_in and
the checks from nav.php and then
sets is_logged_in to true as if you're
viewing your profile, you should be
logged in.



Task Screenshots

Gallery Style: 2 Columns

4 2 1



When I tried to go
to profile.php, it
redirected me to
login.php and
displayed the error
message.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's
being shown*

End of Task 1

Task

100%

Group: Basic Security Rules and Roles
Task #2: Authorization Screenshots
Weight: 20%

Weight: ~20 %
Points: ~0.40

^ COLLAPSE ^

Details:

Include uid/date code comments in all screenshots (one per screenshot is sufficient)



Columns: 3

Sub-Task

100%

Group: Basic
Security Rules and
Roles
Task #2:
Authorization
Screenshots
Sub Task #1:

Sub-Task

100%

Group: Basic
Security Rules and
Roles
Task #2:
Authorization
Screenshots
Sub Task #2:

Sub-Task

100%

Group: Basic
Security Rules and
Roles
Task #2:
Authorization
Screenshots
Sub Task #3:

Task Screenshots

Gallery Style: 2 Columns

4 2 1



has_role function in
user_helpers.php

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's
being shown

Task Response Prompt

Explain in concise steps how this
logically works

Response:

This function takes a parameter of a role (string) and checks if the user is logged in and has a session, from there it checks within the session and sees what name the role the user has is. If the roll name matches the one specified, the function will return true but if the user doesn't have that role, it will return false.

Task Screenshots

Gallery Style: 2 Columns

4 2 1



a PHP if statement
inside of nav.php
that uses
has_roles. It is used
within nav,
assign_roles,
list_roles and
create_roles.

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's
being shown

Task Response Prompt

Explain in concise steps how this
logically works

Response:

This uses the has_role function with the parameter "Admin" to check if a user has the admin role. If the user has it, they will be able to access create_role, list_roles and

Task Screenshots

Gallery Style: 2 Columns

4 2 1



When trying to

access `admin/list_roles.php`,
it redirected me to
login.php and sent
these two error
messages.

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's
being shown

create_role, list_roles and
assign_roles.

End of Task 2

Task



Group: Basic Security Rules and Roles
Task #3: Screenshots of UserRoles and Roles Tables
Weight: ~20%
Points: ~0.40

^ COLLAPSE ^

Details:

Ensure left panel or database name is present in each table screenshot (should contain your ucid)



Columns: 2

Sub-Task



Group: Basic Security Rules and Roles
Task #3: Screenshots of UserRoles
and Roles Tables
Sub Task #1: UserRoles table with at
least one valid entry (Table should
have id, user_id, role_id, is_active,
created, and modified columns)

Sub-Task



Group: Basic Security Rules and Roles
Task #3: Screenshots of UserRoles
and Roles Tables
Sub Task #2: Roles table with at least
one valid entry (Table should have id,
name, description, is_active, modified,
and created columns)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Here is the UserRoles table
shown on my database with
two entries listed, proper
columns are included.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Here is a screenshot of my
Roles table with an entry for
the Admin role and the
correct columns.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 3

Task

Group: Basic Security Rules and Roles

100%

Task #4: Explain how Roles and UserRoles tables work in conjunction with the Users table
 Weight: ~20%
 Points: ~0.40

^ COLLAPSE ^

Columns: 2

Sub-Task

100%

Group: Basic Security Rules and Roles
 Task #4: Explain how Roles and UserRoles tables work in conjunction with the Users table
 Sub Task #1: UserRoles

Sub-Task

100%

Group: Basic Security Rules and Roles
 Task #4: Explain how Roles and UserRoles tables work in conjunction with the Users table
 Sub Task #2: Roles

≡ Task Response Prompt

What's the purpose of the UserRoles table?

Response:

The UserRoles table uses the UserID number from the Users table to map a role to a certain user and then defines what role it is and if it is on or off. By using the UserID number, a shared piece of information can be used to simply assign/unassign roles for the website.

≡ Task Response Prompt

How does Roles.is_active differ from UserRoles.is_active?

Response:

The Roles table provides the roles that the UserRoles table looks to map to users. This is where the roleID in UserRoles comes from and the table also has the role name and if the role is active (turned on/off). This table enables people to create roles and assign them easily by giving them a certain roleID number which is then used in UserRoles.

End of Task 4

Task

100%

Group: Basic Security Rules and Roles
 Task #5: Include pull request links related to this feature
 Weight: ~20%
 Points: ~0.40

^ COLLAPSE ^

i Details:

Should end in /pull/#



⇒ Task URLs

URL #1

<https://github.com/PeterRedfern/par36-it202-007/pull/50>

URL

<https://github.com/PeterRedfern/par36-it202-007>

URL #2

<https://github.com/PeterRedfern/par36-it202-007/pull/53/>

URL

<https://github.com/PeterRedfern/par36-it202-007>

End of Task 5

End of Task 3

End of Group: Basic Security Rules and Roles

Task Status: 5/5

Group



Group: User Profile

Tasks: 5

Points: 2

^ COLLAPSE ^

Task



Group: User Profile

Task #1: View Profile Website Page

Weight: ~20%

Points: ~0.40

^ COLLAPSE ^

Details:

Thoughtful CSS should be applied to all parts (must differ from the "ugly" CSS given via the lessons). The Heroku dev URL must be present in all screenshots of the site.



Sub-Task



Group: User Profile

Task #1: View Profile Website Page

Sub Task #1: Show the profile form correctly populated on page load (username, email) Form should have the following fields: username, email, current password, new password, confirm password (or similar)

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Shows profile.php will the user info populated upon entering the page.

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

End of Task 1

Task



Group: User Profile

Task #2: Explain the logic step-by-step of how the data is loaded and populated when the profile page is visited

Weight: ~20%

Points: ~0.40

^ COLLAPSE ^

i Details:

Don't just show code, translate things to plain English in concise steps.

May be worthwhile using a list.



≡ Task Response Prompt

Response:

1. Once the page is loaded, the PHP script has email and username variables defined and has them equal to `get_user_email` and `get_username` respectively.
2. From there a POST request is made to get the data and display it locally on the webpage.
3. The request gets the data from the database and populates the email and username forms, making sure NOT to populate the current password form by lack of code/variable for it.
4. Now the data is displayed on the webpage for the user to see and interact with.

End of Task 2

Task



Group: User Profile

Task #3: Edit Profile Website Page

Weight: ~20%

Points: ~0.40

^ COLLAPSE ^

i Details:

Thoughtful CSS should be applied to all parts (must differ from the "ugly" CSS given via the lessons). The Heroku dev URL must be present in all screenshots of the site.



Columns: 1

Sub-Task



Group: User Profile

Task #3: Edit Profile Website Page

Sub Task #1: (Two Screenshots) Demonstrate with before and after of a username change (including success message)

📄 Task Screenshots

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Shows a username change update success with the first message (as well as a password reset)

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: User Profile

Task #3: Edit Profile Website Page

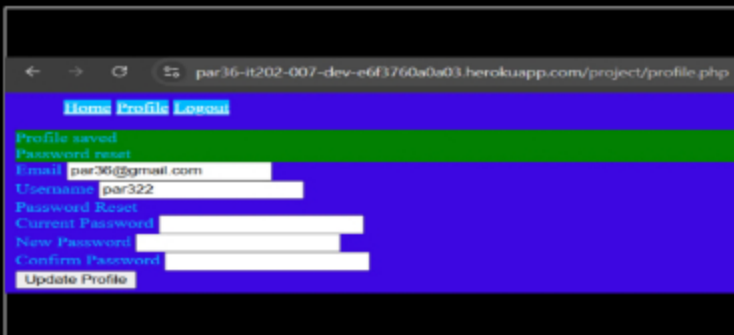
Sub Task #2: Demonstrate the success message of updating password

100%

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Shows a password reset success message, as well as a profile update message.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: User Profile

Task #3: Edit Profile Website Page

Sub Task #3: Demonstrate all JavaScript user-friendly validation messages [can be combined] (email format, username format, password format, new password matching confirm password)

100%

Task Screenshots

Gallery Style: 2 Columns

4 2 1





Shows all JavaScript validation messages

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: User Profile



Task #3: Edit Profile Website Page

Sub Task #4: Demonstrate PHP user-friendly validation message (desired email is already in use)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Message telling the user that the chosen email isn't available with PHP.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: User Profile



Task #3: Edit Profile Website Page

Sub Task #5: Demonstrate PHP user-friendly validation message (Desired username is already in use)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



PHP message that tells the user that the current username isn't available.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: User Profile

Task #3: Edit Profile Website Page

Sub Task #6: Demonstrate PHP user-friendly validation message (Current password doesn't match what's in the DB)

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Tells the user that the current password is invalid in PHP.

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 3

Task

100%

Group: User Profile

Task #4: Explain the logic step-by-step of how the data is checked and saved for the following scenarios

Weight: ~20%

Points: ~0.40

^ COLLAPSE ^

i

Details:

Don't just show code, translate things to plain English

⋮

↓

Columns: 1

Sub-Task

100%

Group: User Profile

Task #4: Explain the logic step-by-step of how the data is checked and saved for the following scenarios

Sub Task #1: Updating Username/Email

⇒ Task Response Prompt

Explain in concise steps how this logically works

Response:

The forms take the data entered by a user for the new username/email and enter it into the database. First it checks for a correct password/new password and if the password checks out, then it will update the username/email data that is in the Users database.

Sub-Task

Group: User Profile

100%

Task #4: Explain the logic step-by-step of how the data is checked and saved for the following scenarios

Sub Task #2: Updating password

⇒ Task Response Prompt

Explain in concise steps how this logically works

Response:

The password is updated when the user enters the current password and a new password two times to confirm it. If the current password is wrong, the request will not go through, also if the two fields for new passwords don't match, it will also not go through. However, if everything lines up, the new password will become the current password in the Users table in the database. Also, before the new password is added in to the database, it is hashed for extra security.

End of Task 4

Task

100%

Group: User Profile

Task #5: Include pull request links related to this feature

Weight: ~20%

Points: ~0.40

^ COLLAPSE ^

i Details:

Should end in /pull/#



⇒ Task URLs

URL #1

<https://github.com/PeterRedfern/par36-it202-007/pull/34>

URL

<https://github.com/PeterRedfern/par36-it202-007>

URL #2

<https://github.com/PeterRedfern/par36-it202-007/pull/41>

URL

<https://github.com/PeterRedfern/par36-it202-007>

URL #3

<https://github.com/PeterRedfern/par36-it202-007/pull/59>

URL

<https://github.com/PeterRedfern/par36-it202-007>

100%

Task #2: Screenshot of your project board from GitHub (tasks should be in the proper column)
 Weight: ~33%
 Points: ~0.33

^ COLLAPSE ^

Task Screenshots

Gallery Style: 2 Columns

4

2

1

Shows project board with all completed tasks in the proper column.

End of Task 2

Task

100%

Group: Misc
 Task #3: Provide a direct link to the project board on GitHub
 Weight: ~33%
 Points: ~0.33

^ COLLAPSE ^

Task URLs

URL #1

<https://github.com/users/PeterRedfern/projects/1>

URL

<https://github.com/users/PeterRedfern/projects/>

End of Task 3

End of Group: Misc
Task Status: 3/3

End of Assignment